

Methods and Properties of Console Class in C#

In this article, I am going to discuss the **Methods and Properties of Console Class in C#** with Examples. Please read our previous article where we discussed the [Basic Structure of a C# Program](#). As part of this article, I am going to discuss the following pointers related to the Console class in detail.

1. [What is Console Class in C#?](#)
2. [Properties of Console Class in C#.](#)
3. [Methods of Console Class in C#.](#)
4. [Understanding the use of the Write and WriteLine method in C#.](#)
5. [Program to show how to print the value of a variable in a Console Application.](#)
6. [Understanding the use of the ReadLine, ReadKey, and Read Methods in C#.](#)
7. [Program to show the use of BackgroundColor, ForegroundColor, and Title properties and Beep Method of the Console class.](#)

What is Console Class in C#?

In order to implement the user interface in console applications, Microsoft provided us with a class called Console. The Console class is available in the System namespace. This Console class provides some methods and properties using which we can implement the user interface in a console application.

In other words, if we want to work with the console window either for taking user input or to show the output, we are provided with the Console in C#.

According to Microsoft documentation the Console class represents the standard input, output, and error streams for console applications and this class cannot be inherited because it is a static class i.e. declared as static as shown in the below image.

```
// Summary:  
//     Represents the standard input, output, and error streams for console applications.  
//     This class cannot be inherited.  
public static class Console  
{
```

The static class in C# contains only static members i.e. all the Properties and Methods available in the Console class are static. So, we can access all these members by using the Console class name i.e. we don't require the Console class instance to access these members.

Properties of Console Class in C#:

There are many properties available in the Console class. Some of them are as follows:

1. **Title**: It gets or sets the title to display in the console title bar. It returns the string to be displayed in the title bar of the console. The maximum length of the title string is 24500 characters.
2. **BackgroundColor**: It gets or sets the background color of the console. It returns a value that specifies the background color of the console; that is, the color that appears behind each character. The default is black.

3. **ForegroundColor**: It gets or sets the foreground color of the console. It returns a ConsoleColor that specifies the foreground color of the console; that is, the color of each character that is displayed. The default is gray.
4. **CursorSize**: It gets or sets the height of the cursor within a character cell. It returns the size of the cursor expressed as a percentage of the height of a character cell. The property value ranges from 1 to 100.

Methods of Console Class in C#:

There are many methods available in the Console class. Some of them are as follows:

1. **Clear()**: It is used to clear the console buffer and corresponding console window of display information. In simple words, it is used to clear the screen.
2. **Beep()**: This method plays the sound of a beep through the console speaker. That means it plays a beep sound using a PC speaker at runtime.
3. **ResetColor()**: This method is used to set the foreground and background console colors to their defaults.
4. **Write("string")**: This method is used to write the specified string value to the standard output stream.
5. **WriteLine("string")**: This method is used to write the specified string value, followed by the current line terminator, to the standard output stream. That means this method same as the write method but automatically moves the cursor to the next line after printing the message.
6. **Write(variable)**: This method is used to write the value of the given variable to the standard output stream.
7. **WriteLine(variable)**: This method is used to write the value of the given variable to the standard output stream along with moving the cursor to the next line after printing the value of the variable.
8. **Read()**: This method read a single character from the keyboard and returns its ASCII value. The Datatype should be int as it returns the ASCII value.
9. **ReadLine()**: This method reads a string value from the keyboard and returns the entered value only. As it returns the entered string value so the DataType is going to be a string.
10. **ReadKey()**: This method reads a single character from the keyboard and returns that character information like what key has been entered, and what its corresponding ASCII value is. The Datatype should be ConsoleKeyInfo which contains the entered key information

Example to show the use of the Write and WriteLine method in C#:

The following example code is self-explained, so please go through the comment lines.

```
//Program to show the use of WriteLine and Write Method
//First Import the System namespace as the
//Console class belongs to System namespace
using System;
namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            //We can access WriteLine and Write method using class name
            //as these methods are static
            //WriteLine Method Print the value and move the cursor to the next
```

```

        Console.WriteLine("Hello");
        //Write Method Print the value and stay in the same line
        Console.Write("HI ");
        //Write Method Print the value and stay in the same line
        Console.Write("Bye ");
        //WriteLine Method Print the value and move the cursor to the next
        Console.WriteLine("Welcome");
        //Write Method Print the value and stay in the same line
        Console.Write("C#.NET ");
        Console.ReadKey();
    }
}

```

Output:

```

Hello
HI Bye Welcome
C#.NET

```

Example to show how to print the value of a variable in C#.

In the below example, I am showing the different ways to print the value of a variable in the C# language.

```

//Program to show how to print the value of a variable
using System;
namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string name = "Pranaya";
            Console.WriteLine(name);
            Console.WriteLine("Hello " + name);
            Console.WriteLine($"Hello {name}");
            Console.ReadKey();
        }
    }
}

```

Output:

```

Pranaya
Hello Pranaya
Hello Pranaya

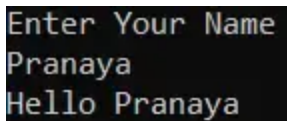
```

Reading Value from the user in C#:

Now, let us understand how to read the value from the user in a console application using the C# language. Here, we are going to use the ReadLine() method to read the values at runtime. The following example shows how to read the value at runtime in a console application in C# using the ReadLine method.

```
//Program to show how to read value at runtime
using System;
namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            //Giving one message to the user to enter his name
            Console.WriteLine("Enter Your Name");
            //ReadLine method reads a string value from the keyboard
            string name = Console.ReadLine();
            //Printing the entered string in the console
            Console.WriteLine($"Hello {name}");
            Console.ReadKey();
        }
    }
}
```

Output:



```
Enter Your Name
Pranaya
Hello Pranaya
```

How do you Read Integer Numbers in C# from the keyword?

Whenever we entered anything whether a string or numeric value from the keyword using the ReadLine method, the input stream is taking it as a string. So, we can directly store the input values in a string variable. If you want to store the input values in integer variables, then we need to convert the string values to integer values. For a better understanding, please have a look at the below example. Here, we are asking the user to enter two integer numbers, and then we are taking those integer numbers as strings, and then we are converting the string into integers and then we are adding those two integers, and showing the result on the console window.

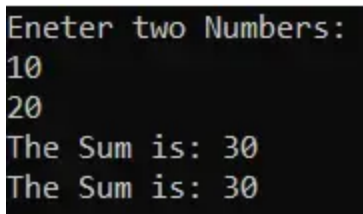
```
//Program to show how to read integer values
using System;
namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
```

```

        Console.WriteLine("Enter two Numbers:");
        //Converting string to Integer
        int Number1 = Convert.ToInt32(Console.ReadLine());
        //Converting string to Integer
        int Number2 = Convert.ToInt32(Console.ReadLine());
        int Result = Number1 + Number2;
        Console.WriteLine($"The Sum is: {Result}");
        Console.WriteLine($"The Sum is: {Number1 + Number2}");
        Console.ReadKey();
    }
}

```

Output:



```

Enter two Numbers:
10
20
The Sum is: 30
The Sum is: 30

```

Note: The ReadLine method always accepts the value in the form of a string. So, we need to convert the values to the appropriate type. In the above example, we are converting the values to integer type by using **Convert.ToInt** method. We will discuss this method in detail in our upcoming articles.

Example to Understand ReadKey and Read method in C#:

The Read method will allow you to enter a character and will return the ASCII value of that character. The ReadKey method also allows you to enter a key and it will return the key information such as what key you pressed, what is the ASCII value of that key, etc. For a better understanding, please have a look at the following example.

```

using System;
namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter a Key");
            int var1 = Console.Read();
            Console.WriteLine($"ASCII Value of the Entered Key is: {var1}");
            Console.WriteLine("Enter Another Key");
            ConsoleKeyInfo var2 = Console.ReadKey();
            Console.WriteLine($"
Entered Key: {var2.Key} KeyChar:{var2.KeyChar}
{(int)var2.KeyChar}");
            Console.WriteLine("Press Any Key to Terminate");
            Console.ReadKey();
        }
    }
}

```

```
}  
}
```

Output:

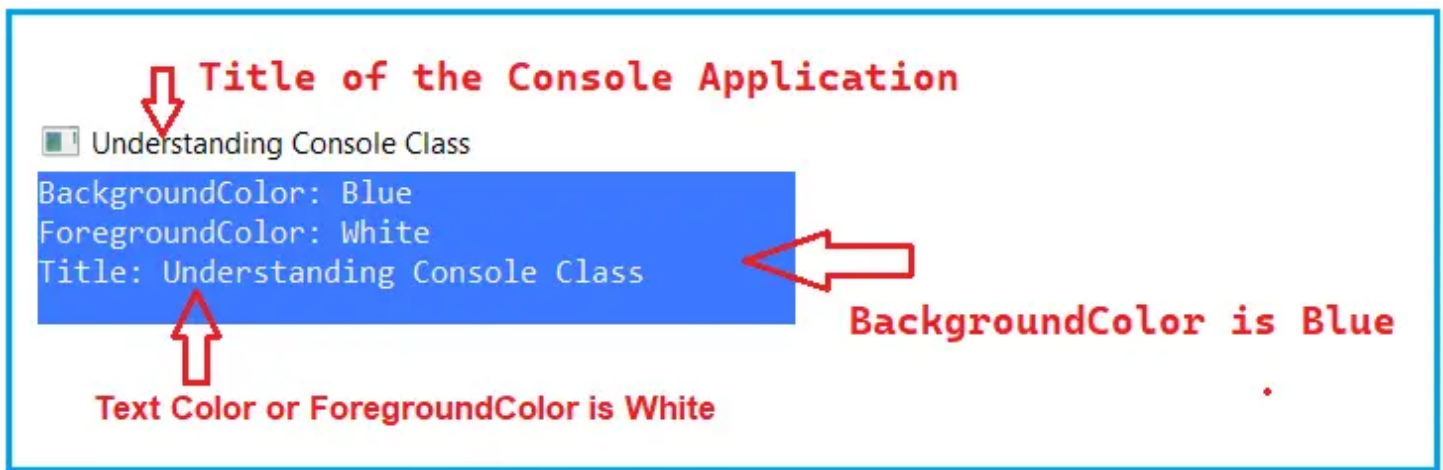
```
Enter a Key  
A  
ASCII Value of the Entered Key is: 65  
Enter Another Key  
B  
Entered Key: B KeyChar:B ASCII:66  
Press Any Key to Terminate
```

Example to understand Console Class Properties:

Now, we will write one program to show the use of BackgroundColor, ForegroundColor, Beep, and Title properties of the Console class in C#. The BackgroundColor property set the background color console and the ForegroundColor property set the text color. The Title property is used to set the Title of the Console Application and the Beep method is used to make a beep sound by using the computer speaker. For a better understanding, please have a look at the below example.

```
//Program to show the use of Console Class Properties and Beep Method  
using System;  
namespace MyFirstProject  
{  
    internal class Program  
    {  
        static void Main(string[] args)  
        {  
            Console.BackgroundColor = ConsoleColor.Blue;  
            Console.ForegroundColor = ConsoleColor.White;  
            Console.Title = "Understanding Console Class";  
            Console.WriteLine("BackgroundColor: Blue");  
            Console.WriteLine("ForegroundColor: White");  
            Console.WriteLine("Title: Understanding Console Class");  
            //It will make Beep Sound  
            Console.Beep();  
            Console.ReadKey();  
        }  
    }  
}
```

Output:



Complex Example to Understand Console Class:

Now, we will write one program to accept Employee Details like EmployeeId, Name, Salary, Address, and Department, and then we will print the accepted information on the console window.

```
//Program to show the use of Console Class
using System;
namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            //Ask User to Enter the Employee Details
            Console.WriteLine("Enter Employee Details");
            Console.WriteLine("Enter Employee ID");
            int EmployeeID = Convert.ToInt32(Console.ReadLine());
            Console.WriteLine("Enter Employee Name");
            string Name = Console.ReadLine();
            Console.WriteLine("Enter Employee Salary");
            int Salary = Convert.ToInt32(Console.ReadLine());
            Console.WriteLine("Enter Employee Address");
            string Address = Console.ReadLine();
            Console.WriteLine("Enter Employee Department");
            string Department = Console.ReadLine();
            //Display the Entered Employee Details
            Console.WriteLine("\nEntered Employee Details are as Follows:");
            Console.WriteLine($"Employee ID Is: {EmployeeID}");
            Console.WriteLine($"Employee Name Is: {Name}");
            Console.WriteLine($"Employee Salary Is: {Salary}");
            Console.WriteLine($"Employee Address Is: {Address}");
            Console.WriteLine($"Employee Department Is: {Department}");
            Console.ReadKey();
        }
    }
}
```

Output:

```
Enter Employee Details
Enter Employee ID
1001
Enter Employee Name
Pranaya
Enter Employee Salary
10000
Enter Employee Address
BBSR
Enter Employee Department
IT

Entered Employee Details are as Follows:
Employee ID Is: 1001
Employee Name Is: Pranaya
Employee Salary Is: 10000
Employee Address Is: BBSR
Employee Department Is: IT
```

Example to Show to Student Mark using Console Class Methods:

Write a Program to enter the Student Registration Number, Name, Mark1, Mark2, Mark3, and calculate the total mark and average marks and then print the student details in the console.

```
using System;
namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            //Ask the user to Enter Student Details
            Console.WriteLine("Enter Student Details");
            Console.WriteLine("Enter Registration Number");
            int RegdNumber = Convert.ToInt32(Console.ReadLine());
            Console.WriteLine("Enter Name");
            string Name = Console.ReadLine();
            Console.WriteLine("Enter Marks of three Subjects:");
            Console.WriteLine("Subject1");
            int Mark1 = Convert.ToInt32(Console.ReadLine());
            Console.WriteLine("Subject2");
            int Mark2 = Convert.ToInt32(Console.ReadLine());
            Console.WriteLine("Subject3");
            int Mark3 = Convert.ToInt32(Console.ReadLine());
            int TotalMarks = Mark1 + Mark2 + Mark3;
            int AverageMark = TotalMarks / 3;
        }
    }
}
```

```

        //Display the Student Details
        Console.WriteLine("\nStudent Details are as Follows:");
        Console.WriteLine($"Registration Number: {RegdNumber}");
        Console.WriteLine($"Name: {Name}");
        Console.WriteLine($"Total Marks : {TotalMarks}");
        Console.WriteLine($"Average Mark: {AverageMark}");
        Console.ReadKey();
    }
}

```

Output:

```

Enter Student Details
Enter Registration Number
10001
Enter Name
Pranaya
Enter Marks of three Subjects:
Subject1
90
Subject2
60
Subject3
75

Student Details are as Follows:
Registration Number: 10001
Name: Pranaya
Total Marks : 225
Average Mark: 75

```

That's it for today. In the next article, I am going to discuss the [Data Types in C#](#) with Examples. Here, in this article, I try to explain the methods and properties of **Console Class in C#** with Examples. And I hope you enjoy this Console Class Methods and Properties in C# with Examples article.